

Advanced Software Engineering — Semester Project Guide

This guide gets you from zero to a working MVP in the first two weeks, and explains how the project will be graded, which AI tools you're allowed to use, and how to pick a tech stack. Keep this doc open through Week 2.

1. The big idea

You will build one software product, solo or in a team of 2–3, and keep it alive for the whole semester. In Weeks 1–2 you'll "vibe code" a rough MVP fast, with heavy AI assistance, before we've formally studied any software engineering principles. Every week after that, we'll study a new principle from the textbook and you'll come back and apply it to your *own* running project — architecture, testing, security, documentation, deployment, all of it.

By Week 15 you'll present a before/after story: here's what I vibe-coded in Week 2, here's what it looks like now, and here's what changed and why.

If your project turns into something genuinely interesting — a good technical story about human+AI collaboration, or a solid case study in applying SE principles to an AI-assisted build — it's a strong candidate for a short paper submitted to the KSPAI conference in January, in coordination with the Research Methodology course.

2. Team structure

- **Solo or teams of 2–3.** Team size should roughly match project scope — a 2–3 person team should attempt something with more moving parts than a solo project.
 - Teams form by end of Week 2. Mixed skill levels on a team are fine and expected; you'll assign roles based on strengths.
-

3. AI tool policy

You do not need to pay for anything. Free AI tools are fully sufficient for this course, and your grade does not depend on which tool you use. What matters is what you understood, decided, and can explain — not how much an AI tool typed for you.

3.1 What's required

1. **An AI Collaboration Log**, updated at every milestone (template below). For each deliverable, note: which tool(s) you used, what you asked for, what you kept vs. rejected/changed, and anything the AI got wrong that you had to fix. This is graded on honesty and reflection, not on how little AI you used.
2. **You must be able to explain every line in your codebase.** The final presentation includes a live, unscripted code walkthrough — pick a file, explain what it does and why it's built that way. This is the real equalizer: it rewards understanding regardless of which tool wrote the first draft.

3.2 Suggested free tools (pick what fits you — none of these are required)

Tool	Type	Notes
GitHub Copilot (Free tier)	IDE completions + chat	Check whether GitHub Education / Classroom50 unlocks a better student tier for you
Google Gemini Code Assist	IDE completions + chat	Very generous free daily quota; strong pick if you want zero friction
Cursor (Free/Hobby)	AI-native editor	Great on-ramp if you want a full AI-first editor experience; rate-limited
Windsurf (formerly Codeium)	IDE completions	Unlimited free autocomplete
Claude / ChatGPT (free web tier)	Chat	Good for planning, architecture discussion, debugging conversations alongside whatever writes your code
Continue / Aider / Cline (open source)	IDE extension / CLI	Bring your own API key (can be a free-tier key); more setup, more control
Local model via Ollama (e.g., a free open-weight coding model)	Fully local	\$0 forever, fully private, no account needed — the true zero-cost-and-zero-dependency option

Tool landscapes shift fast — if you find something better than what's listed here, use it and just log it.

3.3 Licensing & authorship

Briefly: code an AI tool generates for you is a tool output, not a citation — you don't need to "cite" your AI assistant the way you'd cite a paper, but you do need to disclose usage in your Collaboration Log, and you're fully responsible for every line you ship, including its license compatibility if you pull in AI-suggested third-party packages. We'll discuss this properly in Week 14.

4. Choosing a tech stack

Pick based on what you already know a little of, and what your project idea needs. A few well-trodden, free-to-deploy paths:

Stack	Good for	Free hosting
Next.js / React	Interactive apps, dashboards, anything with real client-side state	Vercel
Astro	Content-heavy sites with light interactivity	Vercel, Netlify
Jekyll	Simple static sites, blogs, documentation-style products	GitHub Pages
WordPress	Content management, plugin-driven products	Free tiers vary — ask before committing

Stack	Good for	Free hosting
Flask / Django (Python)	Data-heavy backends, anything closer to a script than a site	Render, Railway (free tiers)
Plain HTML/CSS/JS	Small, fast, no-framework tools	GitHub Pages, Netlify

If you're unsure, default to Next.js or Astro deployed on Vercel — the fastest path from idea to a public URL, and the stack most AI tools are best trained on.

Quick self-check before Week 2

- Which language are you most comfortable reading and debugging (not just generating)?
- Does your idea need a database? (If yes, look at Supabase or a simple SQLite file before reaching for something heavier.)
- Does your idea need real user accounts/auth? (If not, don't add it — scope kills projects.)

5. Project idea menu

You're free to pitch your own idea — these are here to get you unstuck, the way a professor once handed me and my classmates a list that included "a map viewer using TerraServer" (this was before Google Maps existed) and "a system for splitting and uploading large files" (this was before Google Drive existed). The point isn't novelty — it's picking something real enough to keep working on for 15 weeks.

Campus & student life

- A room/study-space finder for your building, using data you collect yourself
- A lost-and-found board scoped to your department
- A study-group matcher based on course schedules

Personal productivity

- A minimalist "read-it-later" tool that strips ads/clutter from saved articles
- A habit or screen-time tracker with a genuinely honest (not gamified) dashboard
- A combined Pomodoro timer + reflective journal

Data & API mashups

- A weather + public-transit trip planner for your city
- A book/movie recommender built on a free public API
- Your own mini version of a research-paper reading aggregator (arXiv + RSS feeds, filtered by topic) — a smaller cousin of tools like sci.aaron.kr

Physical AI / IoT-adjacent (if you have hardware access)

- A live dashboard for ESP32/MQTT sensor data
- A simple visual flow-builder inspired by Node-RED, scoped down
- A viewer for edge-AI (e.g., Jetson) inference results/logs

Language & accessibility tools

- A flashcard app for vocabulary in two languages you're learning/teaching
- A simple OCR helper for scanned documents in a script you care about
- A pronunciation practice tool with recorded playback comparison

Marketplace / directory clones, scoped small

- A simple booking/scheduling tool for a club, lab, or shared space
- A "small file drop" tool for sharing files with an expiring link
- A directory/review site for something under-covered in your city (parks, study cafes, etc.)

6. Milestones (matches the course schedule)

Week	Deliverable
2	Team formed, idea pitched, tech stack chosen
3	Vibe-coded MVP live + product vision statement
4	Retrofit personas & user stories
5	Retrofit lightweight Agile/Scrum process onto ongoing work
6	Architecture diagram + critique of current design
7	Redeploy to a proper cloud host; midterm review
8	Midterm
9	Microservices/decomposition discussion (apply only if it fits your project)
10	Security audit
11	Code style pass
12	Real test suite + peer code review
13	Documentation pass (README, API docs, architecture decision records) + CI setup
14	Final polish, licensing/AI-authorship discussion, presentation rehearsal
15	Final presentation — before/after story + live code walkthrough

7. AI Collaboration Log template

Copy this into your repo as `AI_LOG.md` and add an entry per milestone.

```
## [Milestone name] – [Date]

**Tool(s) used:**
**What I asked for:**
**What I kept as-is:**
**What I changed or rejected, and why:**
**Something the AI got wrong that I had to catch:**
```

8. How this gets graded

The rubric rewards **understanding and process**, not feature count or visual polish:

- Can you explain and defend every decision in your codebase?
- Does your AI Collaboration Log show honest, reflective tool use?
- Did you actually apply each week's SE principle, or just bolt on a feature?
- Is your before/after story genuinely a *before and after*, not just an *after*?

A smaller, well-understood, well-documented project built with free tools will outscore a flashier project the team can't explain.